

Fundamentos: ¿Qué es el Testing Automatizado?

Testing Automatizado



Testing Automatizado

Son pruebas escritas en código para verificar que otra parte de tu aplicación funciona **exactamente** como se espera.

- ✓ Reemplaza la verificación manual en el navegador.
- ✓ Da confianza para refactorizar código sin romperlo.
- ✓ Funciona como documentación viva del proyecto.

El Foco de Hoy: Prueba Unitaria



Una prueba pequeña, **rápida y aislada** sobre una **única unidad de código** (en Vue, un componente individual).

Ejemplo: Probar que un botón cambia de color al recibir un prop, aislando el resto de la app.

Tipos de Pruebas en el Frontend

Existen diferentes niveles para probar una aplicación. Hoy nos enfocaremos exclusivamente en la base.

Tipo de Prueba	¿Qué evalúa?	Velocidad / Aislamiento
Unitarias (Unit)	Componentes individuales aislados del resto del sistema.	 Muy rápidas / Totalmente aisladas.
Integración	Cómo interactúan varios componentes o partes del sistema entre sí.	 Velocidad media / Parcialmente aisladas.
E2E (End-to-End)	Flujos completos simulando a un usuario real en un navegador real.	 Lentas / Nada aisladas (conectan a DB/APIs).

Nuestro Objetivo: Dominar las **Pruebas Unitarias** para asegurar que los componentes de Vue sean robustos antes de conectarlos a integraciones más complejas.

¿Por qué probar Componentes Vue?

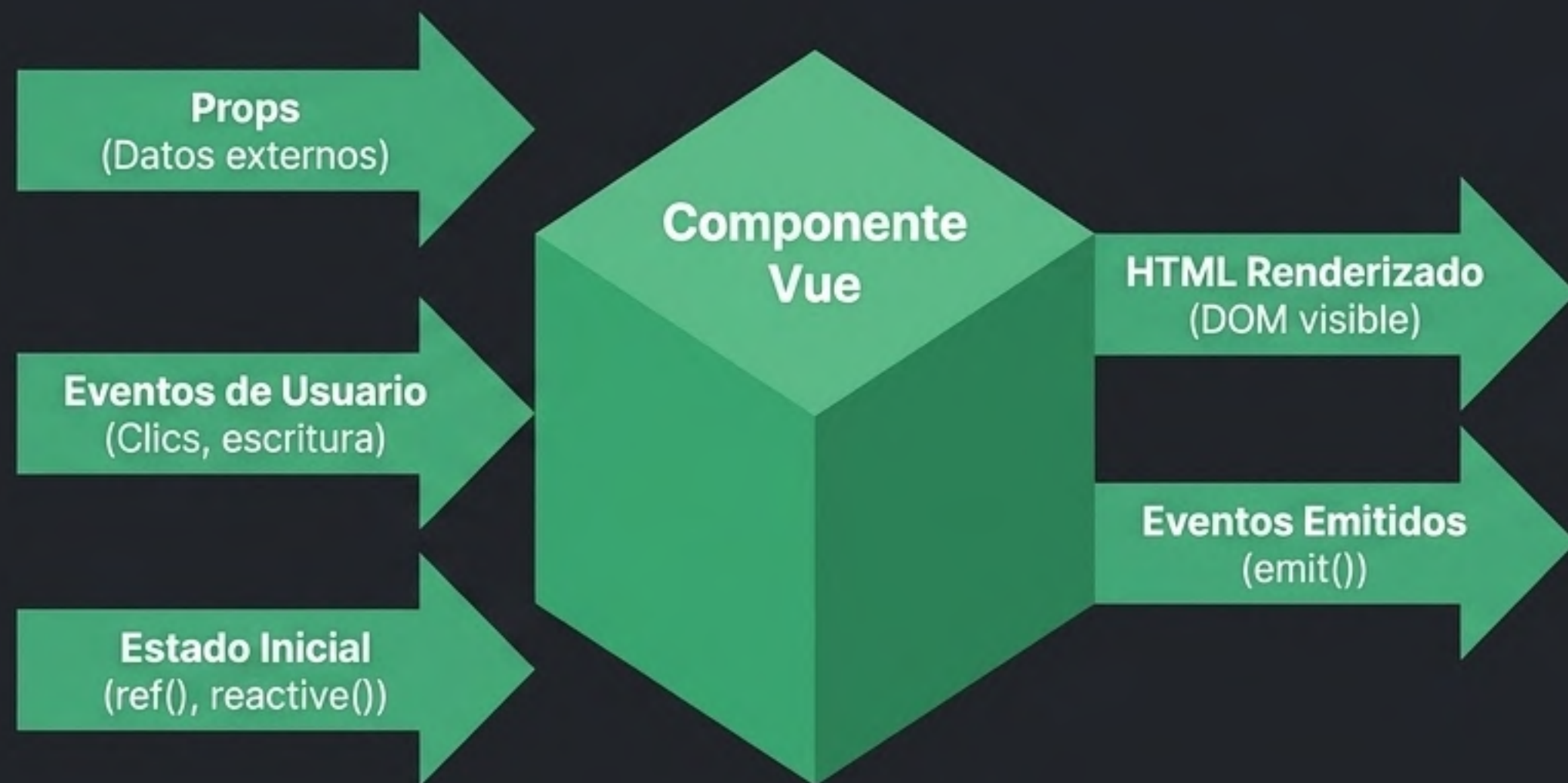
Un componente moderno no es solo una plantilla estática. Es una pequeña máquina lógica.

Debemos probar cómo reacciona nuestro componente ante diferentes escenarios:

🔄 **Estado Reactivo:** ¿Qué pasa cuando una variable `ref()` cambia?

💻 **Eventos:** ¿Qué sucede al escribir en un `<input>`?

☑️ **Renderizado Condicional:** ¿Aparece el mensaje de error cuando falla el `v-if`?



Regla de oro: Prueba el comportamiento (lo que ve y hace el usuario), no los detalles de implementación internos.

Metodología: ¿Qué es TDD?

TDD (Test-Driven Development) es pensar en el resultado esperado antes de escribir la solución.



- **Paso 1: Rojo (Falla).** Escribe una prueba para la funcionalidad que aún no existe. Ejecuta el test y observa cómo falla (esto confirma que la prueba es válida y está conectada).
- **Paso 2: Verde (Pasa).** Escribe el código Vue mínimo y más simple necesario para que la prueba pase con éxito.
- **Paso 3: Refactorizar.** Mejora el código (limpia, optimiza) con la seguridad de que si rompes algo, la prueba te avisará de inmediato.

Herramientas Modernas para Vue 3

Dejamos atrás Jest y Vue CLI para abrazar herramientas nativas impulsadas por Vite.

Vitest

(El Test Runner)

El motor que busca, ejecuta tus archivos de test y reporta los resultados (🔴/🟢).

- ✓ Comparte la misma configuración de Vite (extremadamente rápido).
- ✓ Proporciona las funciones globales de testeo (describe, it, expect).

Vue Test Utils

(La Herramienta de Montaje)

La librería oficial para renderizar componentes Vue en un entorno de pruebas aislado.

- ✓ Permite interactuar con el componente (hacer clics, llenar formularios).
- ✓ Inspecciona el DOM virtual resultante (clases, textos, v-model).

Instalación y Scripts Básicos

Preparar tu entorno impulsado por Vite es rápido.

```
bash

pnpm add -D vitest @vue/test-utils jsdom ← (Necesario para simular un navegador en la consola de Node).
```

```
package.json ×

{
  "scripts": {
    "test:unit": "vitest"
  }
}
```

⚡ Modo Watch: Al ejecutar `pnpm run test:unit`, Vitest buscará archivos `*.test.js` o `*.spec.js`. En desarrollo, mantiene un Watch Mode activo por defecto, re-ejecutando pruebas automáticamente al guardar.

La Anatomía de un Test con Vitest

Todo test se basa en la estructura universal AAA: Arrange (Preparar), Act (Actuar), Assert (Afirmar).

test.spec.ts ×

```
1 import { describe, it, expect } from 'vitest'
2
3 describe('Utilidad de suma', () => {
4   it('suma dos números correctamente', () => {
5     // Arrange & Act
6     const resultado = 2 + 2
7
8     // Assert
9     expect(resultado).toBe(4)
10   })
11 })
```

Agrupar pruebas relacionadas. Funciona como una caja o suite de pruebas.

Define un caso de prueba individual. Su nombre debe leerse como una oración (Ej: eso... suma números).

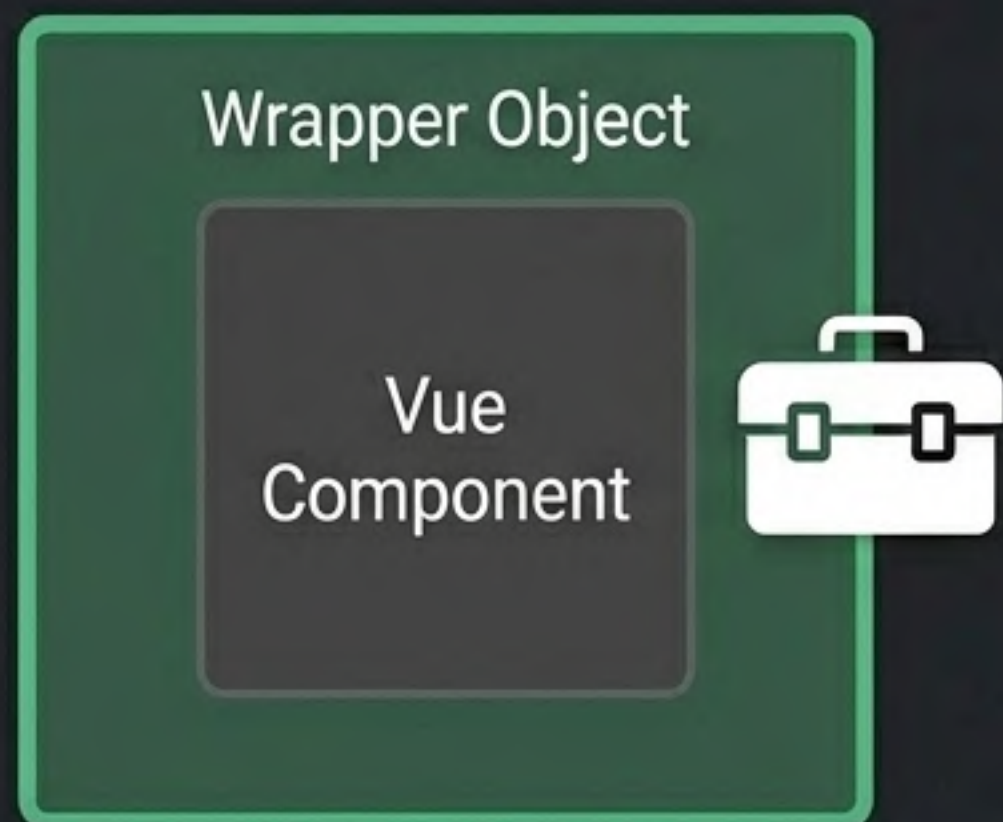
La aserción. Es donde declaras lo que esperas que sea el resultado final.

Montar un Componente con Vue Test Utils

Para probar un componente, primero necesitamos renderizarlo en memoria usando mount.

```
import { mount } from '@vue/test-utils'
import LoginForm from '@components/LoginForm.vue'

const wrapper = mount(LoginForm)
```



¿Qué es el wrapper?

`mount()` no devuelve HTML plano. Devuelve un **Envoltorio** que contiene el componente Vue renderizado más una caja de herramientas de métodos para interactuar con él:

- `wrapper.find()`: Buscar elementos.
- `wrapper.trigger()`: Simular eventos (clics).
- `wrapper.setValue()`: Escribir en inputs.
- `wrapper.exists()`: Verificar si se renderizó.

Selectores Estables: data-testid

Depender de clases CSS para buscar elementos hace que los tests sean frágiles.

❌ El Camino Frágil (Clases CSS)

```
<button class="btn btn-blue">  
Ingresar</button>
```

```
// En el test:  
wrapper.find('.btn-blue')
```

Si el equipo de diseño cambia la clase a **btn-red** por estética, ¡el test fallará aunque la lógica funcione perfectamente!

✅ El Camino Sólido (data-testid)

```
<button data-testid="submit">  
Ingresar</button>
```

```
// En el test:  
wrapper.find('[data-testid="submit"]')
```

El atributo **data-testid** es **invisible** para el usuario y el CSS. Deja claro a otros desarrolladores que ese elemento está anclado a un test y no debe alterarse.

Primera Prueba: Render Inicial

Comprobemos el estado estático de LoginForm.vue antes de que el usuario interactúe.

```
import { mount } from '@vue/test-utils'
import { describe, it, expect } from 'vitest'
import LoginForm from '@components/LoginForm.vue'

describe('LoginForm', () => {
  it('renderiza campos y botón deshabilitado al inicio', () => {
    // 1. Arrange: Montar el componente
    const wrapper = mount(LoginForm)

    // 2. Assert: Verificar que los inputs existen
    expect(wrapper.find('[data-testid="email"]').exists()).toBe(true)

    // 3. Assert: Verificar el estado del botón
    expect(wrapper.find('[data-testid="submit"]').attributes('disabled')).toBeDefined()
  })
})
```

.attributes() devuelve un objeto con los atributos HTML del elemento. Si el botón tiene disabled, estará definido.

Prueba Dinámica: Llenando el Formulario

Simular entrada de usuario con setValue requiere manejar la naturaleza asíncrona de Vue.

```
// Nota el uso de async 📌
it('habilita botón con datos válidos', async () => {
  const wrapper = mount(LoginForm)

  // await asegura que Vue procese el v-model y actualice el DOM
  await wrapper.find('[data-testid="email"]').setValue('test@mail.com')
  await wrapper.find('[data-testid="password"]').setValue('123456')

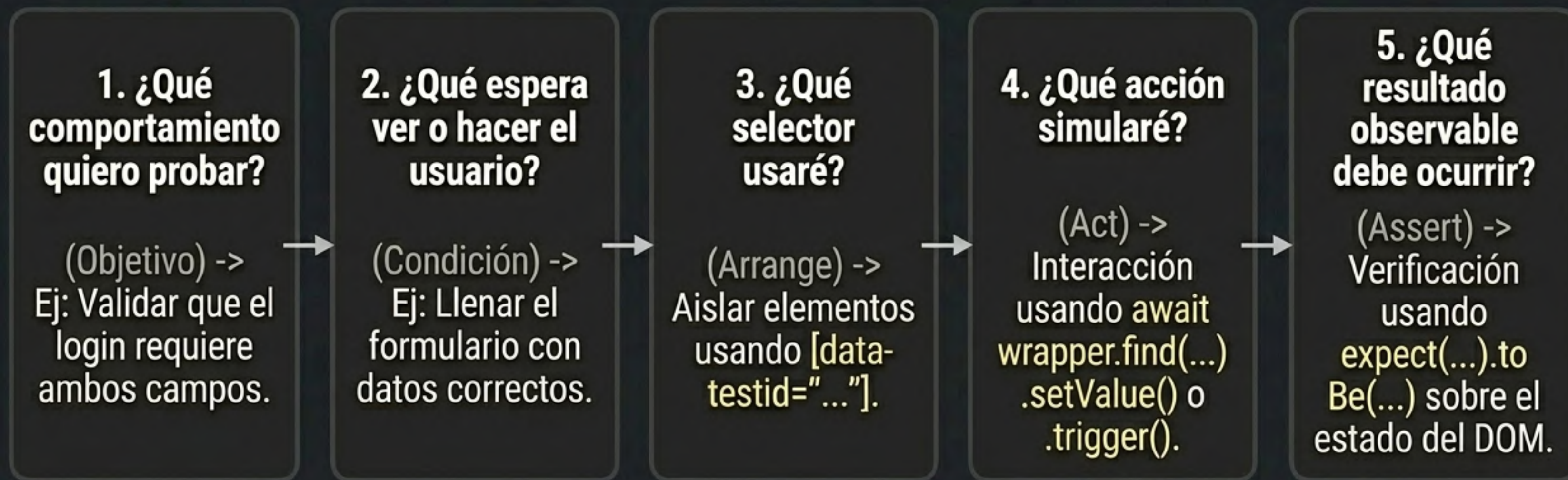
  // Verificamos que el botón ya no está bloqueado
  expect(wrapper.find('[data-testid="submit"]').attributes('disabled')).toBeUndefined()
})
```

⚠️ ¿Por qué usar await?

Actualizar el DOM en Vue es una operación asíncrona. Si no esperas (**await**) a que **setValue** termine su trabajo, tu aserción (**expect**) se ejecutará antes de que Vue quite el atributo **disabled**, causando un falso negativo.

Mini-Guía de Estudio: Diseñando tu Test

Antes de escribir código, hazte estas 5 preguntas para estructurar tus pruebas con éxito.



Día 1 Completado. Ya dominas el ciclo básico. En el **Día 2**, profundizaremos en probar eventos personalizados (**emitted**), simular dependencias (mocks) y configurar el enrutador.